SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION COURSE CODE:ITA0510 Slot A

COURSE OUTCOME COVERED

CO3: ***Analyze and extract image features using detection and matching techniques for effective image representation and comparison.*** **(BTL 4)**

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks 50** **Duration :60 Minutes Annexure A**

Case Study

Code of Conduct:

I _____P.Jyothika(192311156)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **P.Jyothika**

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	

Clarity	**10**	**Clear, well-structured, and easy to understand**	**Organized and understandable presentation**	**Limited clarity and structure**	**Poor clarity; difficult to understand**	

1. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail.

Analyse the problem and explain how preprocessing enhances edge detection results.

Problem Analysis

In a low-contrast image, the intensity difference between an object and its background is very small. Edge detection algorithms such as Sobel, Prewitt, and Canny depend on intensity changes to identify boundaries. When contrast is low, the gradient magnitude becomes weak, so important edges may not be detected.

Effect of Low Contrast

- Object and background have similar intensity values.
- Gradient strength becomes weak.
- Important edges may disappear.
- Threshold-based detectors may classify real edges as non-edges.
- The resulting edge map becomes incomplete.

Preprocessing Methods

1. **Contrast Enhancement:** Histogram Equalization or CLAHE (Contrast Limited Adaptive Histogram Equalization) increases the difference between dark and bright regions.
2. **Noise Reduction:** Gaussian or median filtering can remove noise before edge detection. This prevents false edges.
3. **Image Normalization:** Pixel intensities can be normalized to a wider range, improving gradient calculation.
4. **Adaptive Thresholding:** Instead of using one fixed threshold, adaptive thresholds can be used to detect weak edges in different regions.

Recommended Workflow

Input Image → Noise Removal → Contrast Enhancement → Normalization → Canny/Sobel → Edge Map

Analysis

For a low-contrast image, directly applying Canny may produce missing or weak edges. Applying CLAHE followed by Gaussian filtering and Canny detection generally provides a better balance between enhancing true boundaries and suppressing noise.

Solution

A suitable approach is:

Low Contrast Image → CLAHE → Gaussian Blur → Canny Edge Detection This improves edge visibility and produces a more complete edge map.

Conclusion

Preprocessing is essential for low-contrast images because it strengthens intensity differences before edge detection. Proper contrast enhancement and noise reduction significantly improve the accuracy and continuity of detected edges.

2. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive key points. Analyse the issue and suggest methods to improve feature selection.

Problem Analysis

Highly textured regions contain many small intensity variations. Corner detectors such as Harris Corner Detector and Shi-Tomasi interpret strong local intensity changes as potential corners. Therefore, textured regions can generate a very large number of key points. This creates excessive or redundant key points, increasing computational cost and making later feature matching more difficult.

Problems Caused by Excessive Key points

- Too many redundant features.
- Increased computational complexity.
- More false or unstable key points.
- Difficult feature matching.

- Reduced efficiency of object recognition or tracking.

Feature Selection Methods

1. Non-Maximum Suppression (NMS):

Keeps only the strongest keypoint within a local neighborhood.

2. Response Thresholding:

Discard corners whose detector response is below a selected threshold.

3. Minimum Distance Constraint:

Ensure detected key points are separated by a minimum distance.

4. Adaptive Thresholding:

Use different thresholds depending on local texture complexity.

5. ANMS (Adaptive Non-Maximal Suppression):

Selects strong and spatially distributed key points instead of allowing them to cluster in one region.

6. Limit Number of Key points:

Retain only the top N strongest and most reliable features.

Recommended Workflow

Input Image → Corner Detection → Response Calculation → Thresholding → NMS/ANMS → Selected Keypoints

Analysis

Simply increasing the detection threshold may remove excessive points, but it can also eliminate useful features. ANMS combined with response thresholding is better because it retains strong features while ensuring spatial distribution.

Solution

Use:

- Harris/Shi-Tomasi detector
- Response threshold
- Non-maximum suppression
- Minimum key point distance
- ANMS for spatial distribution

Conclusion

The main issue is not that corner detection is incorrect, but that highly textured areas produce too many valid responses. Intelligent feature selection produces fewer, stronger, and more spatially distributed key points.

3. Feature Matching with Noise

Feature matching between two images fails due to noise in one image. Analyse the impact of noise and explain how preprocessing improves matching accuracy.

Problem Analysis

Feature matching identifies corresponding points between two images. If one image contains noise, the local intensity patterns around key points can change significantly. This can affect both feature detection and descriptor computation.

For example, salt-and-pepper noise can create artificial edges and corners, while Gaussian noise changes pixel intensities around existing features.

Impact of Noise

- Creates false key points.
- Changes the appearance of true key points.
- Makes descriptors less reliable.
- Increases incorrect matches.
- Reduces the number of correct matches.
- Can cause matching algorithms to fail. **Preprocessing Methods**

1. Gaussian Filtering:

Reduces Gaussian noise while preserving general image structure.

2. Median Filtering:

Very effective for salt-and-pepper noise and preserves edges better than simple averaging.

3. Bilateral Filtering:

Reduces noise while preserving important edges.

4. Contrast/Intensity Normalization:

Reduces differences caused by illumination and intensity variations.

5. Robust Feature Detection:

Use detectors such as SIFT or similar robust local-feature methods when appropriate.

Matching Improvement

After preprocessing:

Noisy Image → Denoising → Feature Detection → Descriptor Extraction → Descriptor Matching → Ratio Test/RANSAC

A ratio test can reject ambiguous descriptor matches, while RANSAC can remove geometrically inconsistent matches.

Analysis

Noise affects matching at multiple stages. If noisy pixels are used to create descriptors, even a correct corresponding point may produce very different descriptors in the two images. Therefore, preprocessing should occur before feature extraction, rather than trying to correct all errors after matching. Solution

A good solution is:

Median/Gaussian Filtering → Robust Feature Detector → Robust Descriptor → Ratio Test → RANSAC

Conclusion

Noise reduces feature-matching accuracy by changing local image structures and descriptors. Appropriate denoising before feature extraction, combined with robust matching techniques, significantly improves the number and reliability of correct matches.

4. PCA and Information Loss

A system applies PCA aggressively and loses important feature details.

Analyse the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

Problem Analysis

Principal Component Analysis (PCA) is a dimensionality-reduction technique that transforms original features into a smaller set of principal components.

The first components contain the largest amount of variance. However, if PCA removes too many components, information contained in lower-variance dimensions may also be lost.

Trade-off

More Components	Fewer Components
More information retained	More information lost
Higher dimensionality	Lower dimensionality
Higher computational cost	Lower computational cost
Better reconstruction	Poorer reconstruction
May contain redundancy	More compact representation

The objective is therefore not to minimize the number of components, but to find the smallest number that preserves sufficient useful information.

Explained Variance

PCA provides an explained variance ratio for each component.

The cumulative explained variance is:

Cumulative Variance = Sum of explained variance ratios

For example:

Components	Cumulative Variance
10	75%
20	88%
30	94%
40	97%
50	99%

If the application requires approximately 95% information retention, around 30 components would be a reasonable choice in this example. **Methods for Optimal Selection**

1. Explained Variance Threshold:

Select components that preserve approximately 90–99% variance depending on the application.

2. Scree Plot:

Look for the elbow point, where additional components provide relatively little improvement.

3. Cross-Validation:

Evaluate the actual downstream task, such as classification accuracy, for different component counts.

4. Reconstruction Error:

Select a number of components that keeps reconstruction error within an acceptable limit.

Analysis

Aggressive PCA may improve speed and reduce storage but can remove discriminative information. Importantly, high variance does not always mean high predictive importance. Therefore, selecting components only according to a very low dimension can hurt classification or recognition performance.

Solution

A practical method is:

Original Features → Standardization → PCA → Explained Variance Analysis → Select Optimal Components → Downstream Model

Choose the component count based on both explained variance and actual model performance.

Conclusion

PCA involves a trade-off between computational efficiency and information preservation. The optimal number of components should retain sufficient variance while maintaining the performance of the final application.

5. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection.

Analyse the issue and explain how appropriate feature descriptors improve matching performance.

Problem Analysis

In this case, feature detection works correctly, but matching fails because the descriptor is inappropriate for the detected features.

A feature detector identifies where important points are located, while a descriptor represents what the local region around each point looks like. Therefore, successful detection alone does not guarantee successful matching.

Detector vs Descriptor

Component	Purpose
Feature Detector	Finds important points
Feature Descriptor	Represents the local appearance
Matcher	Compares descriptors between images

For example, Harris can successfully detect corners, but the choice of descriptor determines how reliably those corners can be matched.

Problems with Incorrect Descriptors

- Poor representation of local image structure.
- Sensitivity to rotation or scale.
- High number of false matches.
- Low number of correct matches.
- Descriptor distance becomes unreliable.

Appropriate Descriptor Selection

1. SIFT Descriptor
    - Robust to scale changes.
    - Robust to rotation.
    - Useful for general image matching.
    - Uses a floating-point descriptor.
2. ORB Descriptor
    - Fast and computationally efficient.
    - Suitable for real-time applications.
    - Uses a binary descriptor.

3. BRIEF

- Very fast.
- Suitable for simple matching situations.
- Less robust to rotation unless modified.

4. HOG

- Represents local gradient/orientation information.
- Useful for shape-related applications, though it is not a general replacement for local keypoint descriptors.

Descriptor-Matcher Compatibility

The distance measure should match the descriptor type:

- SIFT/SURF-type floating-point descriptors → Euclidean/L2 distance
- ORB/BRIEF-type binary descriptors → Hamming distance

Using the wrong distance measure can significantly reduce matching accuracy.

Recommended Workflow

Image → Feature Detection → Descriptor Extraction → Appropriate Matcher → Ratio Test → RANSAC → Correct Matches

For example:

Harris Detector + suitable local descriptor + appropriate distance metric + RANSAC or

ORB Detector/Descriptor + Hamming Matcher

Analysis

The system's failure occurs because feature detection and feature description are separate stages. Even if key points are accurately detected, an unsuitable descriptor may produce similar representations for unrelated points or different representations for corresponding points.

Solution

Select the descriptor based on:

- Scale changes
- Rotation
- Illumination changes
- Computational requirements
- Image type

- Descriptor data type

Then use a compatible distance metric and reject ambiguous matches using the ratio test and RANSAC.

Conclusion

Feature matching performance depends strongly on descriptor quality and descriptor–matcher compatibility. Selecting an appropriate descriptor for the image conditions, followed by robust matching and geometric verification, provides more accurate and reliable feature correspondence.